FRED Calc Add-In ××× Build & Install

FRED Calc Add-In — build & install

A LibreOffice Calc add-in (UNO component, **Java**) exposing FRED economic-data functions as worksheet formulas:

Function	Signature	Returns
`FRED_SERIES`	`FRED_SERIES(series_id; [start_date]; [end_date]; [api_key]; [headers])`	spillable 2-column array `(date, value)`
`FRED_DESCRIPTION`	`FRED_DESCRIPTION(series_id; [api_key])`	series title
`FRED_META`	`FRED_META(series_id; field; [api_key])`	one metadata field
`FRED_FIELDS`	`FRED_FIELDS(series_id; [api_key]; [headers])`	spillable 2-column array `(field, value)` — lists valid `FRED_META` fields
`FRED_LATEST`	`FRED_LATEST(series_id; [api_key])`	most recent value

> In Calc's UI, arguments are separated by **semicolons**: `=FRED_SERIES("GDP"; "2020-01-01"; "2024-01-01")`.
>
> Every function takes a trailing optional `api_key`. Supply it to override the environment variable (see below) — typed literally or as a cell reference: `=FRED_DESCRIPTION("GDP"; $B$1)`.

1. Prerequisites

Windows

1. **LibreOffice + SDK.** Default install path `C:\Program Files\LibreOffice`, with the SDK under `…\LibreOffice\sdk`. The SDK provides `sdk\bin\unoidl-write.exe` and `sdk\bin\javamaker.exe`.
2. **A JDK to build with** — any JDK 8 or newer (`javac`, `jar`). The build targets **Java 8 bytecode** (`--release 8`), so the add-in runs on the Oracle JRE 8 that LibreOffice accepts out of the box. Pass its path with `-Jdk`, or set `JAVA_HOME`, or put `javac` on `PATH`.
3. **Runtime JRE — nothing to change.** The component is Java-8 bytecode and uses only `java.net.HttpURLConnection`, so LibreOffice's existing/default Oracle JRE (8+) runs it as-is. No *Tools* ▸ *Options* ▸ *Advanced* change needed.

> Why not `java.net.http.HttpClient`? That needs Java 11+, but LibreOffice's `javavendors.xml` allow-list here accepts only Oracle/Sun/IBM/Azul/Amazon — the Java 21 runtimes on this machine (JetBrains JBR) are rejected. Targeting Java 8 + `HttpURLConnection` (both JDK standard library) avoids installing a new JRE while still meeting the "JDK standard library only" requirement.

Confirm the tools resolve:

```
& 'C:\Program Files\LibreOffice\sdk\bin\unoidl-write.exe' --help
& 'C:\Program Files\LibreOffice\sdk\bin\javamaker.exe'    # prints usage
& 'C:\Program Files\Android\Android Studio\jbr\bin\javac.exe' -version   # any 8+
```

Linux / macOS

1. **A JDK 8** (`javac`, `jar`). If your distro's package manager has one (`apt install openjdk-8-jdk-headless`, `dnf install java-1.8.0-openjdk-devel`, …) use that. Otherwise, no root needed — fetch a JDK 8 build straight from Eclipse Adoptium/Temurin and unpack it under your home directory:

   ```
   curl -s "https://api.adoptium.net/v3/assets/latest/8/hotspot?architecture=x64&image_type=jdk&os=linux&vendor=eclipse" \
     | grep -o '"link": *"[^"]*tar.gz"' | head -1 | cut -d'"' -f4
   # download that URL, then:
   mkdir -p ~/jdks && tar xzf OpenJDK8U-jdk_x64_linux_hotspot_*.tar.gz -C ~/jdks
   export JAVA_HOME=~/jdks/jdk8u<version>   # match the extracted directory name
   export PATH="$JAVA_HOME/bin:$PATH"
   ```

2. **LibreOffice + SDK.** If your distro packages a matching SDK (`apt install libreoffice-dev libreoffice-dev-common` on

Debian/Ubuntu), use that and skip to confirming the tools below. Otherwise, download the generic Linux tarballs (RPM-based; works on any distro since we only extract the `.rpm`s, not install them) from https://download.documentfoundation.org/libreoffice/stable/, e.g. for 26.2.4/x86_64: `LibreOffice_26.2.4_Linux_x86-64_rpm.tar.gz` and `LibreOffice_26.2.4_Linux_x86-64_rpm_sdk.tar.gz`. Extract each `.rpm` inside them with `rpm2cpio` / `cpio` into a prefix directory — no root required:

```
tar xzf LibreOffice_*_rpm.tar.gz LibreOffice_*_rpm_sdk.tar.gz
mkdir -p ~/opt
for rpm in LibreOffice_*_rpm/RPMS/*.rpm LibreOffice_*_rpm_sdk/RPMS/*.rpm; do
  rpm2cpio "$rpm" | cpio -idm --no-absolute-filenames -D ~/opt
done
mv ~/opt/opt/libreoffice* ~/libreoffice26.2   # adjust to the extracted version
export LO_HOME=~/libreoffice26.2
```

This lays out the same `program/` and `sdk/bin/` tree the Windows install has (`unoidl-write`, `javamaker`, `types.rdb`, `program/classes/*.jar`).

3. **Java vendor allow-list.** LibreOffice only loads a JVM whose `java.vendor` appears in `$LO_HOME/program/javavendors.xml` (Sun, Oracle, IBM, Blackdown, BEA, Azul, Amazon by default). A stock Temurin/Adoptium build reports vendor `Temurin`, which is **not** on that list — `unopkg` will fail with `CannotRegisterImplementationException: Could not create Java implementation loader` when installing the extension. Add an entry for it in your local `javavendors.xml` (this file lives inside your own LibreOffice install, not a system-shared one, so editing it is safe):

```
<vendor name="Temurin">
  <minVersion>1.8.0</minVersion>
</vendor>
```

Insert it next to the other `<vendor>` entries, inside `<vendorInfos>`. (If your JDK came from your distro's package manager, its vendor is usually already on the list and this step is unnecessary.)

Confirm the tools resolve:

```
"$LO_HOME/sdk/bin/unoidl-write"          # prints usage
"$LO_HOME/sdk/bin/javamaker"             # prints usage
"$JAVA_HOME/bin/javac" -version          # any 8+
```

2. Provide the FRED API key (never hardcoded)

Two ways, in priority order:

1. **The `api_key` function argument** — the optional last argument of every function. Type it literally, or (recommended) put it in one cell and reference that cell: `=FRED_LATEST("UNRATE"; $B$1)`. Wins when supplied.
2. **The `FRED_API_KEY` environment variable** — used whenever the argument is omitted (falls back further to the `fred.api.key` Java system property).

Get a free key at https://fredaccount.stlouisfed.org/apikeys. The key is never hardcoded in the add-in. The environment-variable route below is convenient because it keeps the key out of the spreadsheet entirely.

Set it **for the LibreOffice process** — i.e. set it, then launch `soffice` from that same shell (or set it as a persistent user env var and restart LibreOffice):

```
$env:FRED_API_KEY = 'your_32_char_key'
& 'C:\Program Files\LibreOffice\program\soffice.exe'
```

Persistent (survives reboots; restart LibreOffice afterwards):

```
setx FRED_API_KEY "your_32_char_key"
```

Linux / macOS — same idea, `export` instead of `$env:` / `setx`:

```
export FRED_API_KEY='your_32_char_key'
"$LO_HOME/program/soffice"
```

Persistent: add the `export` line to `~/.bashrc` (or `~/.zshrc`), then restart your shell and LibreOffice.

3. Build the .oxt

Windows

From the project root:

```
# JAVA_HOME set, or javac on PATH:
pwsh -File build.ps1
# or point at a specific JDK (e.g. the Android Studio JBR used during testing):
pwsh -File build.ps1 -Jdk 'C:\Program Files\Android\Android Studio\jbr'
```

This runs the full pipeline and produces `build\FRED.oxt`:

```
1. unoidl-write  idl\**              -> build\types\XFred.rdb
2. javamaker     build\types\XFred.rdb -> build\gen\**.class
3. javac         src\**.java         -> build\classes\**.class
4. jar           classes + bindings  -> build\oxt\fred.jar
5. zip           staging tree        -> build\FRED.oxt
```

The equivalent commands by hand

If you'd rather run each step yourself (paths assume the defaults above):

```
$LO = 'C:\Program Files\LibreOffice'
$env:PATH = "$LO\program;$env:PATH"

# 1. IDL -> UNO type library
& "$LO\sdk\bin\unoidl-write.exe" "$LO\program\types.rdb" idl build\types\XFred.rdb

# 2. type library -> Java bindings
& "$LO\sdk\bin\javamaker.exe" -nD -Gc -O build\gen -X "$LO\program\types.rdb" build\types\XFred.rdb

# 3. compile to Java 8 bytecode (URE jars live in program\classes)
javac --release 8 -cp "build\gen;$LO\program\classes\*" -d build\classes (Get-ChildItem src -Recurse -Filter *.java).FullName

# 4. package the component jar (RegistrationClassName comes from the manifest)
jar cfm build\oxt\fred.jar registration\MANIFEST.MF -C build\classes . -C build\gen .

# 5. stage config/types/manifest, then zip the four entries into build\FRED.oxt
#    (types/XFred.rdb, fred.jar, config/CalcAddIns.xcu, description.xml, META-INF/manifest.xml)
```

Linux / macOS

`build.sh` is a POSIX port of `build.ps1` running the identical pipeline. From the project root:

```
export JAVA_HOME=~/jdks/jdk8u<version>   # or wherever your JDK 8 lives
export LO_HOME=~/libreoffice26.2         # or wherever LibreOffice + SDK live
./build.sh
# or pass paths explicitly instead of the env vars:
./build.sh --jdk ~/jdks/jdk8u<version> --libreoffice ~/libreoffice26.2
```

This produces `build/FRED.oxt` via the same five steps as `build.ps1`. Two JDK-8-specific quirks it works around, in case you're compiling by hand:

- `javac --release 8` **doesn't exist on JDK 8 itself** (the flag was added in JDK 9); `build.sh` detects a `1.x` `javac -version` and falls back to `-source 8 -target 8`, which is equivalent for a straight JDK-8 build.
- `jar` **on JDK 8 can reject duplicate directory entries** when packaging two class trees that share a package path (`-C build/classes . -C build/gen .` both contain `com/example/fred/`), throwing `java.util.zip.ZipException: duplicate entry: com/`. `build.sh` merges both trees into one staging directory first, then jars that single tree.

The equivalent commands by hand

```
LO="$LO_HOME"
export PATH="$LO/program:$PATH"

# 1. IDL -> UNO type library
"$LO/sdk/bin/unoidl-write" "$LO/program/types.rdb" idl build/types/XFred.rdb

# 2. type library -> Java bindings
"$LO/sdk/bin/javamaker" -nD -Gc -O build/gen -X "$LO/program/types.rdb" build/types/XFred.rdb

# 3. compile (JDK 9+: --release 8; JDK 8 itself: -source 8 -target 8)
"$JAVA_HOME/bin/javac" -source 8 -target 8 -cp "build/gen:$LO/program/classes/*" \
  -d build/classes $(find src -name '*.java')

# 4. merge classes + bindings (avoids the JDK-8 jar duplicate-entry issue), then jar
mkdir -p build/jarstage
cp -r build/classes/. build/jarstage/
cp -r build/gen/. build/jarstage/
"$JAVA_HOME/bin/jar" cfm build/oxt/fred.jar registration/MANIFEST.MF -C build/jarstage .

# 5. stage config/types/manifest, then zip the four entries into build/FRED.oxt
#    (types/XFred.rdb, fred.jar, config/CalcAddIns.xcu, description.xml, META-INF/manifest.xml)
```

4. Install into LibreOffice

Close LibreOffice first, then use `unopkg` from the SDK/program dir:

```
& 'C:\Program Files\LibreOffice\program\unopkg.exe' add --force build\FRED.oxt
# list / remove:
& 'C:\Program Files\LibreOffice\program\unopkg.exe' list
& 'C:\Program Files\LibreOffice\program\unopkg.exe' remove com.example.fred
```

You can also install by double-clicking `build\FRED.oxt` (opens the Extension Manager). After installing, **restart LibreOffice** from a shell that has `FRED_API_KEY` set (step 2).

Linux / macOS:

```
"$LO_HOME/program/unopkg" add --force build/FRED.oxt
# list / remove:
"$LO_HOME/program/unopkg" list
"$LO_HOME/program/unopkg" remove com.example.fred
```

5. Try it

Open `test\fred_demo.fods`, then recalculate with **Ctrl+Shift+F9**. The "Live result" column calls each function; the "Formula" column shows exactly what to type. Or in any sheet:

```
=FRED_DESCRIPTION("GDP")                      -> Gross Domestic Product
=FRED_META("GDP"; "units")                    -> Billions of Dollars
=FRED_FIELDS("GDP")                           -> spills (field, value) rows
=FRED_LATEST("UNRATE")                        -> e.g. 4.1
=FRED_SERIES("GDP"; "2020-01-01"; "2024-01-01")  -> spills (date, value) rows
```

Behavior notes

- **Errors → Calc error values.** Bad series IDs, unknown metadata fields, a missing API key, or a network failure raise a UNO exception, which Calc shows as an error value (e.g. `#VALUE!`) in the cell — not an exception string.
- **Multi-cell / spilling.** LibreOffice has no dynamic spill: to get all rows of `FRED_SERIES`, select the output range (2 columns × N rows) and enter it as an **array formula** — Ctrl+Shift+Enter, or tick **Array** in the Function Wizard. A single-cell entry shows only the first value.

- **Header row.** Pass `headers` = `TRUE` (or `1`) as the last `FRED_SERIES` argument to prepend a `Date` / `Value` header row, e.g. `=FRED_SERIES("GDP"; "2020-01-01"; ""; ""; TRUE())`. Add one extra row to the selected range for the header.
- **Date arguments.** `start_date` / `end_date` accept either an ISO `YYYY-MM-DD` string or a **date-typed cell** (a numeric date serial is converted using the default 1899-12-30 epoch), so `=FRED_SERIES(B4;B2;B3;B1)` with date cells in `B2` / `B3` works.
- **Missing observations.** FRED's `.` sentinel becomes an empty value cell in `FRED_SERIES`; `FRED_LATEST` skips missing values and returns the most recent numeric one.
- **Per-session caching.** Every raw response is cached by request URL for the life of the office session, so recalculation does not re-hit the API. Restart LibreOffice to clear the cache.
- **No third-party jars.** HTTP uses `java.net.HttpURLConnection`; JSON is parsed by a small hand-rolled parser (`Json.java`). Nothing beyond the JDK + UNO is bundled.

Automated test

`tools\test_fred.py` drives a headless LibreOffice over a UNO socket and checks the core functions plus the error paths against live FRED data (companion scripts `test_apikey.py`, `test_datecells.py`, `test_headers.py`, and `test_fields.py` cover the optional arguments and `FRED_FIELDS`):

```
$env:FRED_API_KEY = 'your_key'
& 'C:\Program Files\LibreOffice\program\soffice.exe' --headless --norestore --accept="socket,host=localhost,port=2002;urp;"
& 'C:\Program Files\LibreOffice\program\python.exe' tools\test_fred.py   # prints RESULT: PASS
```

`tools\test_apikey.py` verifies the optional `api_key` argument in isolation — run it with `FRED_API_KEY` **unset**, passing the key only as an argument:

```
Remove-Item Env:\FRED_API_KEY -ErrorAction SilentlyContinue
& 'C:\Program Files\LibreOffice\program\soffice.exe' --headless --norestore --accept="socket,host=localhost,port=2002;urp;"
& 'C:\Program Files\LibreOffice\program\python.exe' tools\test_apikey.py your_key
```

Linux / macOS — same idea, LibreOffice's bundled `python` (it ships the `uno` module) instead of `python.exe`, run in the background so the shell isn't blocked:

```
export FRED_API_KEY='your_key'
"$LO_HOME/program/soffice" --headless --norestore --accept="socket,host=localhost,port=2002;urp;" &
"$LO_HOME/program/python" tools/test_fred.py   # prints RESULT: PASS
```

`test_apikey.py` verifies the optional `api_key` argument with `FRED_API_KEY` **unset** (`test_datecells.py`, `test_headers.py`, and `test_fields.py` also take the key as an argument). Each test script calls `desktop.terminate()` when it finishes, so relaunch `soffice` before running the next one:

```
unset FRED_API_KEY
"$LO_HOME/program/soffice" --headless --norestore --accept="socket,host=localhost,port=2002;urp;" &
"$LO_HOME/program/python" tools/test_apikey.py your_key
```

Troubleshooting

- `unoidl-write` / `javamaker` "not found" → pass the right `-LibreOffice` path; the SDK must be installed (it is a separate download from LibreOffice).
- Functions show `#NAME?` → the extension isn't registered; confirm with `unopkg list` and restart LibreOffice.
- Every FRED cell is an error value (`Err:502`) → `FRED_API_KEY` isn't set in the process that launched LibreOffice. Set it, then launch `soffice` from that same environment (or `setx` it and fully restart LibreOffice, tray Quickstarter included).
- `unopkg add` fails with `CannotRegisterImplementationException: Could not create Java implementation loader` (Linux/macOS) → your JDK's vendor isn't in `$LO_HOME/program/javavendors.xml`'s allow-list — see the Java vendor allow-list note in Prerequisites.